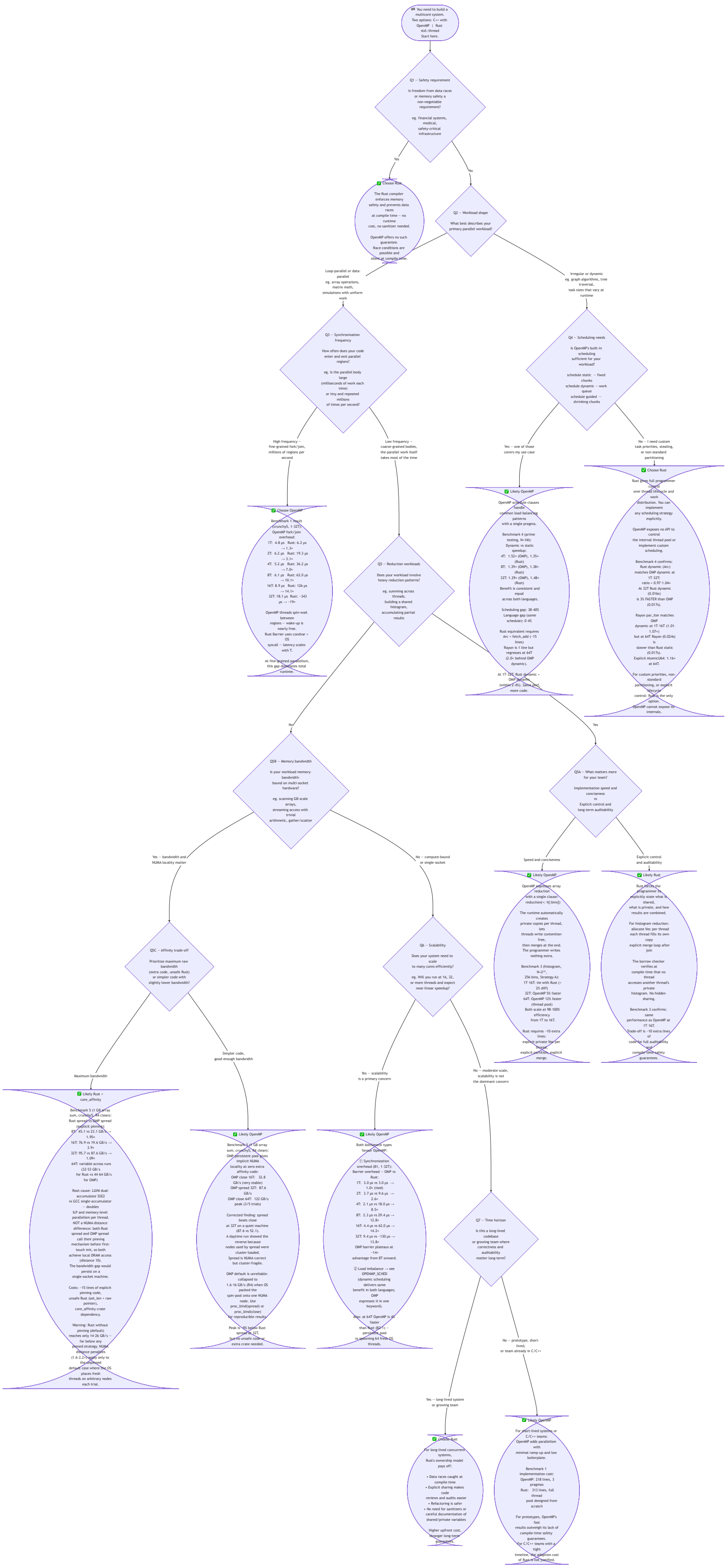
You need to build a multicore system.
Two options: C++ with OpenMP | Rust std::thread
Start here.
Q1 — Safety requirement
Is freedom from data races or memory safety a non-negotiable requirement?
eg. financial systems, medical, safety-critical infrastructure
Yes
No
Choose Rust
The Rust compiler enforces memory safety and prevents data races at compile time — no runtime cost, no sanitizer needed.
OpenMP offers no such guarantee. Race conditions are possible and silent at compile time.
Q2 — Workload shape
What best describes your primary parallel workload?
Loop-parallel or data-parallel
eg. array operations, matrix math, simulations with uniform work
Irregular or dynamic
eg. graph algorithms, tree traversal, task sizes that vary at runtime
Q3 — Synchronization frequency
How often does your code enter and exit parallel regions?
eg. Is the parallel body large (milliseconds of work each time) or tiny and repeated millions of times per second?
High frequency — fine-grained fork/join, millions of regions per second
Low frequency — coarse-grained bodies, the parallel work itself takes most of the time
Q4 — Scheduling needs
Is OpenMP's built-in scheduling sufficient for your workload?
schedule static — fixed chunks
schedule dynamic — work queue
schedule guided — shrinking chunks
Yes — one of those covers my use case
No — I need custom task priorities, stealing, or non-standard partitioning
Choose OpenMP
Benchmark 1 result (crunchy5, 1-32T): OpenMP fork/join overhead:
1T: 4.8 µs Rust: 6.2 µs → 1.3×
2T: 6.2 µs Rust: 19.3 µs → 3.1×
4T: 5.2 µs Rust: 36.2 µs → 7.0×
8T: 6.1 µs Rust: 62.0 µs → 10.1×
16T: 8.9 µs Rust: 126 µs → 14.1×
32T: 18.1 µs Rust: ~343 µs → ~19×
OpenMP threads spin-wait between regions — wake-up is nearly free.
Rust Barrier uses condvar + OS syscall — latency scales with T.
At fine-grained parallelism, this gap dominates total runtime.
Likely OpenMP
OpenMP schedule clauses handle common load-balancing patterns with a single pragma.
Benchmark 4 (prime testing, N=1M): Dynamic vs static speedup:
4T: 1.52× (OMP), 1.35× (Rust)
8T: 1.39× (OMP), 1.38× (Rust)
32T: 1.29× (OMP), 1.48× (Rust)
Benefit is consistent and equal across both languages.
Scheduling gap: 38-40% Language gap (same schedule): 0-4%
Rust equivalent requires Arc + fetch_add (~15 lines)
Rayon is 1 line but regresses at 64T (2.0× behind OMP dynamic).
At 1T-32T, Rust dynamic ≈ OMP dynamic (within 2-4%). Same perf, more code.
Choose Rust
Rust gives full programmer control over thread lifecycle and work distribution. You can implement any scheduling strategy explicitly.
OpenMP exposes no API to control the internal thread pool or implement custom scheduling.
Benchmark 4 confirms: Rust dynamic (Arc) matches OMP dynamic at 1T-32T: ratio = 0.97-1.04×
At 32T Rust dynamic (0.016s) is 3% FASTER than OMP (0.017s).
Rayon par_iter matches OMP dynamic at 1T-16T (1.01-1.07×) but at 64T Rayon (0.024s) is slower than Rust static (0.017s).
Explicit AtomicU64: 1.16× at 64T.
For custom priorities, non-standard partitioning, or explicit lifecycle control: Rust is the only option.
OpenMP cannot expose its internals.
Q5 — Reduction workloads
Does your workload involve heavy reduction patterns?
eg. summing across threads, building a shared histogram, accumulating partial results
No
Yes
Q5B — Memory bandwidth
Is your workload memory-bandwidth-bound on multi-socket hardware?
eg. scanning GB-scale arrays, streaming access with trivial arithmetic, gather/scatter
Yes — bandwidth and NUMA locality matter
No — compute-bound or single-socket
Q5A — What matters more for your team?
Implementation speed and conciseness
vs
Explicit control and long-term auditability
Speed and conciseness
Explicit control and auditability
Likely OpenMP
OpenMP expresses array reduction with a single clause: reduction(+: h[:bins])
The runtime automatically creates private copies per thread, lets threads write contention-free, then merges at the end. The programmer writes nothing extra.
Benchmark 3 (histogram, N=2^, 256 bins, Strategy A): 1T-16T: tie with Rust (< 2% diff)
32T: OpenMP 5% faster
64T: OpenMP 12% faster (thread pool)
Both scale at 98-100% efficiency from 1T to 16T.
Rust requires ~10 extra lines: explicit private Vec per thread, explicit partition, explicit merge.
Likely Rust
Rust forces the programmer to explicitly state what is shared, what is private, and how results are combined.
For histogram reduction: allocate Vec per thread each thread fills its own copy explicit merge loop after join
The borrow checker verifies at compile time that no thread accesses another thread's private histogram. No hidden sharing.
Benchmark 3 confirms: same performance as OpenMP at 1T-16T.
Trade-off is ~10 extra lines of code for full auditability and compile-time safety guarantees.
Q5C — Affinity trade-off
Prioritize maximum raw bandwidth (extra code, unsafe Rust) or simpler code with slightly lower bandwidth?
Maximum bandwidth
Simpler code, good-enough bandwidth
Q6 — Scalability
Does your system need to scale to many cores efficiently?
eg. Will you run at 16, 32, or more threads and expect near-linear speedup?
Yes — scalability is a primary concern
No — moderate scale, scalability is not the dominant concern
Likely Rust + core_affinity
Benchmark 5 (1 GB array sum, crunchy5, R4 clean): Rust spread vs OMP spread (explicit pinning):
8T: 45.1 vs 23.1 GB/s → 1.95×
16T: 76.9 vs 19.6 GB/s → 3.9×
32T: 95.7 vs 87.6 GB/s → 1.09×
64T: variable across runs (32-53 GB/s for Rust vs 44-64 GB/s for OMP)
Root cause: LLVM dual-accumulator SSE2 vs GCC single-accumulator — doubles ILP and memory-level parallelism per thread. NOT a NUMA distance difference: both Rust spread and OMP spread call their pinning mechanism before first-touch init, so both achieve local DRAM access (distance 10).
The bandwidth gap would persist on a single-socket machine.
Costs: ~15 lines of explicit pinning code, unsafe Rust (set_len + raw pointer), core_affinity crate dependency.
Warning: Rust without pinning (default) reaches only 14-26 GB/s — far below any pinned strategy. NUMA distance penalties (1.6-2.2×) apply only to the unpinned default case where the OS places fresh threads on arbitrary nodes each trial.
Likely OpenMP
Benchmark 5 (1 GB array sum, crunchy5, R4 clean): OMP persistent pool gives implicit NUMA locality at zero extra affinity code:
OMP close 16T: 32.8 GB/s (very stable)
OMP spread 32T: 87.6 GB/s
OMP close 64T: 122 GB/s peak (3/5 trials)
Corrected finding: spread beats close at 32T on a quiet machine (87.6 vs 52.1).
A daytime run showed the reverse because nodes used by spread were cluster-loaded.
Spread is NUMA-correct but cluster-fragile.
OMP default is unreliable: collapsed to 1.6-16 GB/s (R4) when OS packed the spin-pool onto one NUMA node. Use proc_bind(spread) or proc_bind(close) for reproducible results.
Peak is ~8% below Rust spread at 32T, but no unsafe code or extra crate needed.
Likely OpenMP
Both bottleneck types favour OpenMP:
① Synchronization overhead (B1, 1-32T): Barrier overhead — OMP vs Rust:
1T: 3.0 µs vs 3.0 µs → 1.0× (tied)
2T: 3.7 µs vs 9.6 µs → 2.6×
4T: 2.1 µs vs 18.0 µs → 8.5×
8T: 2.3 µs vs 29.4 µs → 12.8×
16T: 4.4 µs vs 62.0 µs → 14.2×
32T: 9.4 µs vs ~130 µs → 13.8×
OMP barrier plateaus at ~14× advantage from 8T onward.
② Load imbalance → see OPENMP_SCHED (dynamic scheduling delivers same benefit in both languages; OMP expresses it in one keyword).
Also: at 64T OpenMP is 4× faster than Rust (B2^1) — persistent pool vs spawning 64 fresh OS threads.
Q7 — Time horizon
Is this a long-lived codebase or growing team where correctness and auditability matter long-term?
Yes — long-lived system or growing team
No — prototype, short-lived, or team already in C/C++
Choose Rust
For long-lived concurrent systems, Rust's ownership model pays off:
• Data races caught at compile time
• Explicit sharing makes code reviews and audits easier
• Refactoring is safer
• No need for sanitizers or careful documentation of shared/private variables
Higher upfront cost, stronger long-term guarantees.
Likely OpenMP
For short-lived systems or C/C++ teams: OpenMP adds parallelism with minimal ramp-up and low boilerplate.
Benchmark 1 implementation cost:
OpenMP: 218 lines, 3 pragmas
Rust: 313 lines, full thread pool designed from scratch
For prototypes, OpenMP's fast results outweigh its lack of compile-time safety guarantees.
For C/C++ teams with a tight timeline, the adoption cost of Rust is not justified.